

World-Model Based Reinforcement Learning for Hockey: DreamerV3 with 2-Phase Training Curriculum

Carl Kueschall
University of Tübingen

Winter Semester 2025/26

1 Introduction

Note: Originally a two-person project; co-author exmatriculated. Work completed by remaining author.

The hockey environment [2] presents a challenging continuous control problem in an adversarial setting. Two agents compete in an air-hockey game with 18-dimensional state observations encoding positions, velocities, and puck state, while each player controls 4 continuous action dimensions. Episodes last up to 250 timesteps with extremely sparse rewards: +10 for scoring, −10 for conceding, and 0 otherwise. This sparsity makes credit assignment difficult—agents must learn complex behaviors (positioning, puck control, shooting, defending) from reward signals that occur only at episode boundaries.

We implement DreamerV3 [1], a world-model based reinforcement learning algorithm that addresses sparse rewards by learning entirely in “imagination.” The agent first learns a world model from real experience, then trains its policy through simulated rollouts in learned latent space. Our implementation builds upon the NaturalDreamer codebase¹, with significant modifications for the hockey domain: Two-Hot Symlog encoding for sparse reward prediction, DreamSmooth [3] temporal reward smoothing, mixed opponent training, and a 2-phase curriculum that first saturates performance against fixed bots, then adds self-play from a strong checkpoint to achieve best results.

Contributions: DreamerV3 implementation with Two-Hot Symlog, DreamSmooth, mixed opponent training, and PFSP self-play—achieving 93.5% combined win rate (90% weak, 97% strong) and top pre-tournament leaderboard performance.

2 Methods

2.1 DreamerV3: World-Model Based RL

2.1.1 Background

DreamerV3 [1] learns a world model that predicts future states and rewards from actions, then trains actor-critic policies entirely through imagined rollouts in learned latent space. This is particularly suited for sparse-reward environments: the world model propagates reward information backward through imagined trajectories, enabling credit assignment even when real rewards are rare. Our implementation adapts this for the hockey domain, addressing (1) extremely sparse goal rewards, (2) continuous 4D action space, and (3) adversarial dynamics.

¹<https://github.com/InexperiencedMe/NaturalDreamer>

2.1.2 World Model Architecture

The world model uses a Recurrent State Space Model (RSSM) combining deterministic and stochastic dynamics. The state $s_t = (h_t, z_t)$ consists of a deterministic GRU hidden state $h_t \in \mathbb{R}^{256}$ and stochastic latent z_t (16 categorical variables, 16 classes each). Dynamics: $h_t = \text{GRU}(h_{t-1}, z_{t-1}, a_{t-1})$; prior $\hat{z}_t \sim p_\theta(z_t|h_t)$; posterior $z_t \sim q_\phi(z_t|h_t, \text{Enc}(o_t))$. KL divergence with free nats thresholding trains the model.

Two-Hot Symlog. MSE fails on hockey’s sparse reward distribution $\{-10, 0, +10\}$. We use Two-Hot Symlog encoding [1]: $\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1)$, discretizing into 255 bins. The two-hot encoding places mass on bins surrounding the target, enabling gradient flow. Applied to both reward and value prediction.

2.1.3 Key Modifications for Hockey

Table 1 summarizes our domain-specific modifications. **DreamSmooth** [3] propagates goal rewards backward: $\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$ with $\alpha = 0.5$, addressing temporal credit assignment for goals at episode boundaries.

Modification	Problem Addressed	Implementation
Two-Hot Symlog	MSE fails on sparse rewards	255 bins in symlog space
DreamSmooth	Temporal credit assignment	$\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$, $\alpha = 0.5$
Mixed Opponents	Overfitting to single opponent	50% weak / 50% strong per episode
Self-Play + PFSP	Limited diversity	20-checkpoint pool, variance-weighted selection

Table 1: Key modifications from baseline DreamerV3 for the hockey domain.

2.1.4 Behavior Learning

Actor: Tanh-squashed Gaussian with $\sigma_{\min} = 0.1$. **Critic:** Two-Hot Symlog with slow EMA target ($\tau = 0.98$). **Lambda returns:** $\text{TD}(\lambda)$, $\lambda = 0.95$, $\gamma = 0.997$. **Value normalization:** Percentile-based scaling (5th/95th) for advantages. **Actor loss:**

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{s \sim \text{imagine}} \left[\frac{V^\lambda(s) - V(s)}{S} + \eta \cdot H[\pi(\cdot|s)] \right] \quad (1)$$

with $\eta = 3 \times 10^{-4}$ fixed.

3 Experimental Evaluation

3.1 2-Phase Training Curriculum

We use a 2-phase curriculum (Table 2). **Phase 1** trains against mixed opponents (50% weak, 50% strong) *without* self-play until performance saturates. We select a strong checkpoint at 180k gradient steps and resume in a new run. **Phase 2** adds self-play from episode 1: a pool of 20 past checkpoints with PFSP (variance mode) opponent selection and 50% anchor / 50% pool mix. The same learning rates and replay ratio (32) are used in both phases.

The top panel of Figure 1 shows `eval/combined_win_rate` across both runs. The Phase 1 run (orange) saturates near 95% by ~ 200 k steps. Phase 2 (red) begins at 180k already high; `combined_win_rate`

Phase	Time	RR	LR (Actor)	Opponents	Combined Win Rate
1: Mixed only	~39h	32	1×10^{-4}	Weak + Strong (no self-play)	0% $\rightarrow$ ~95%
2: + Self-Play	~20h	32	1×10^{-4}	Mixed + Pool (20 checkpoints)	~90% $\rightarrow$ 93.5%

Table 2: 2-phase training curriculum. RR = replay ratio. Phase 2 resumes from 180k checkpoint. Best checkpoint at 336k steps.²

does not show dramatic further gain because the metric is evaluated against fixed weak/strong bots, which Phase 1 already mastered. The benefit of self-play is therefore *not* visible in this metric.

3.2 Self-Play Metrics and Ablation Studies

Self-play progress is visible in pool-vs-pool metrics (Figure 1, middle row): win rate vs. oldest third rises 50% $\rightarrow$ 97%; vs. newest third 0% $\rightarrow$ 80%; vs. pool overall 50% $\rightarrow$ 87%. Later checkpoints clearly beat earlier ones. Ablation studies (bottom) validate key modifications: DreamSmooth and Two-Hot Symlog each contribute to final performance.

3.3 Final Benchmark Performance

Table 3 presents evaluation of the best checkpoint (336k steps) over 100 games per opponent with deterministic policy (seed 42). Performance against the strong bot (97%) exceeds the weak bot (90%); we hypothesize the strong bot’s predictable play is easier to exploit than the weak bot’s suboptimal variance.

Opponent	Win Rate	Tie Rate	Loss Rate	Avg Reward
Weak Bot	90%	7%	3%	+8.16
Strong Bot	97%	0%	3%	+8.73
Combined	93.5%	3.5%	3.0%	+8.45

Table 3: Final benchmark (336k checkpoint, 100 games per opponent, deterministic, seed 42).

4 Discussion

Key Findings. Two-Hot Symlog is critical—MSE fails on sparse $\{-10, 0, +10\}$ rewards. DreamSmooth accelerates learning by propagating goal signals backward. The 2-phase curriculum is effective: Phase 1 saturates performance against fixed bots without self-play; Phase 2 adds self-play from a strong checkpoint, and improvement is visible in self-play metrics (win rate vs. pool) rather than in `combined_win_rate` against fixed opponents. Mixed opponent training prevents overfitting to a single bot type. PFSP variance mode balances exposure to diverse opponents.

Limitations. Evaluation is limited to weak and strong bots; the tournament will reveal generalization. ~59h total training constrains hyperparameter search. **Future Work.** Population-based training; model-based planning during evaluation; transfer to other adversarial domains.

5 AI Usage Declaration

IDE Autocomplete: Cursor AI used throughout. **Claude Code:** Learning (DreamerV3 internals, RL debugging) and routine tasks (plotting, W&B, evaluation scripts). Core algorithmic decisions from careful

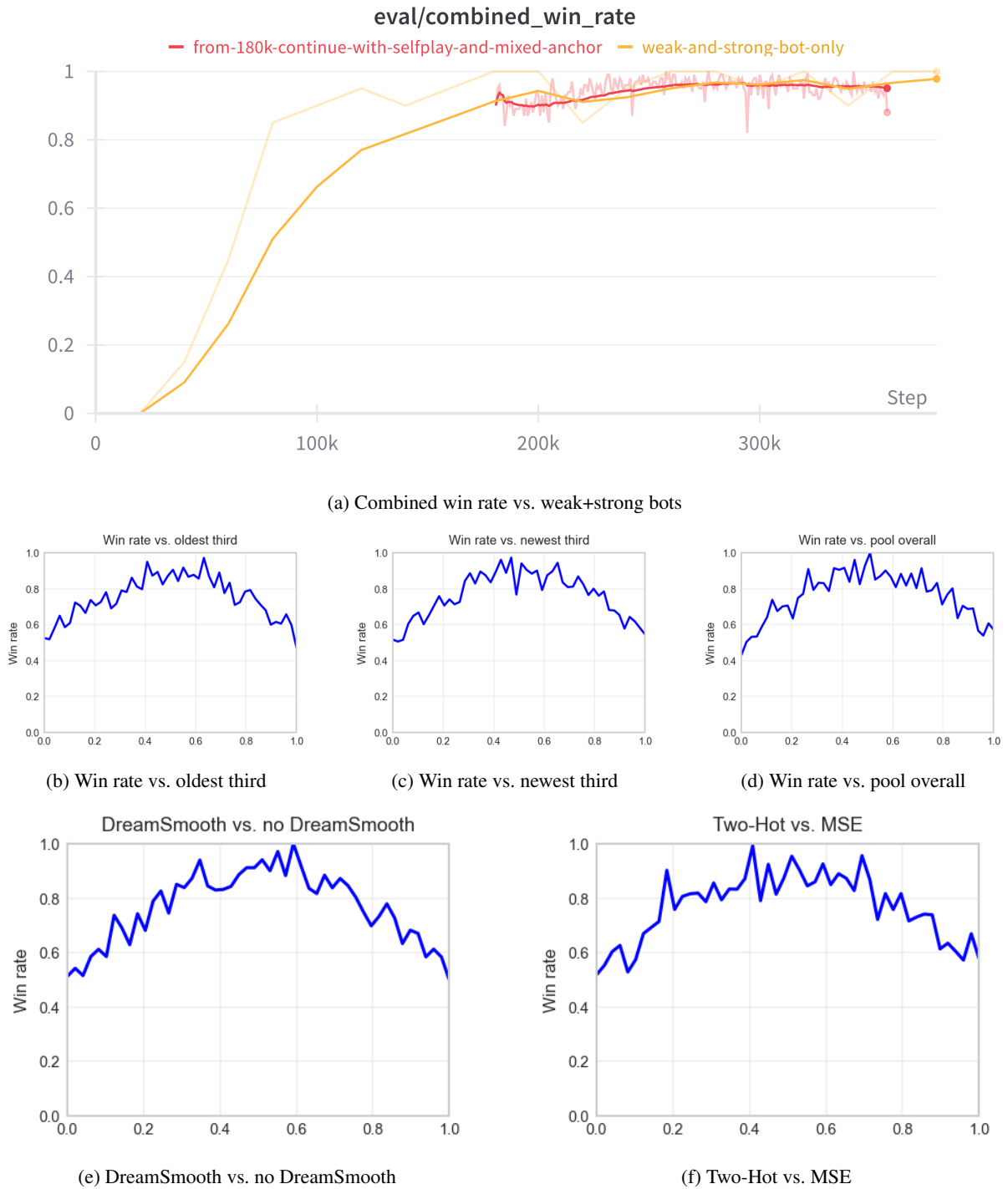


Figure 1: Top: Combined win rate vs. fixed bots across phases. Middle: Self-play metrics (win rate vs. pool thirds). Bottom: Ablation studies (DreamSmooth, Two-Hot). Placeholders replace with actual figures.

analysis.

References

- [1] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2024.
- [2] G. Martius, P. Koley, M. Zhobro, and S. Ramen. Hockey environment. <https://github.com/martius-lab/hockey-env>, 2024.
- [3] V. Wu, J. Aslanides, J. Ba, and D. Hafner. Dreamsmooth: Improving model-based reinforcement learning via reward smoothing. *arXiv preprint arXiv:2311.01450*, 2023.